

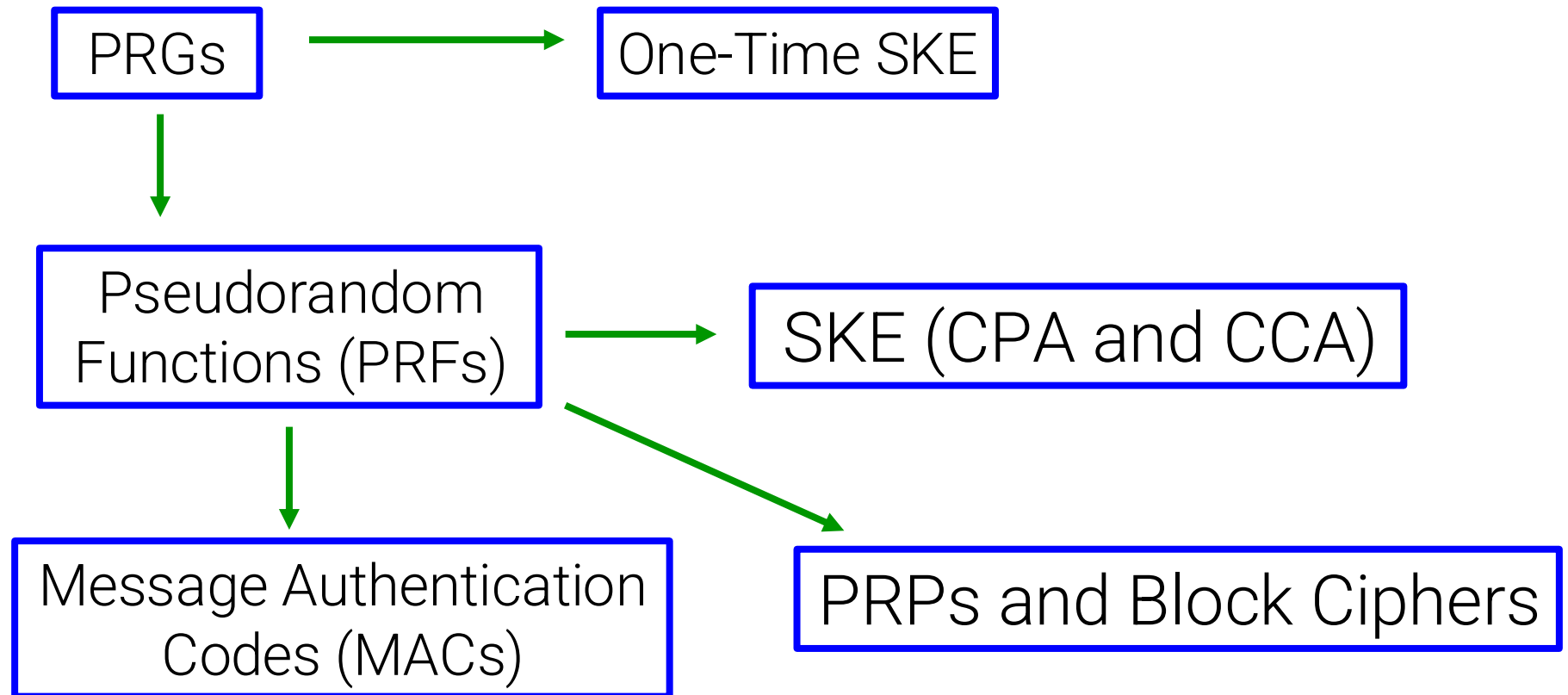
COS 433: Cryptography

Announcements:

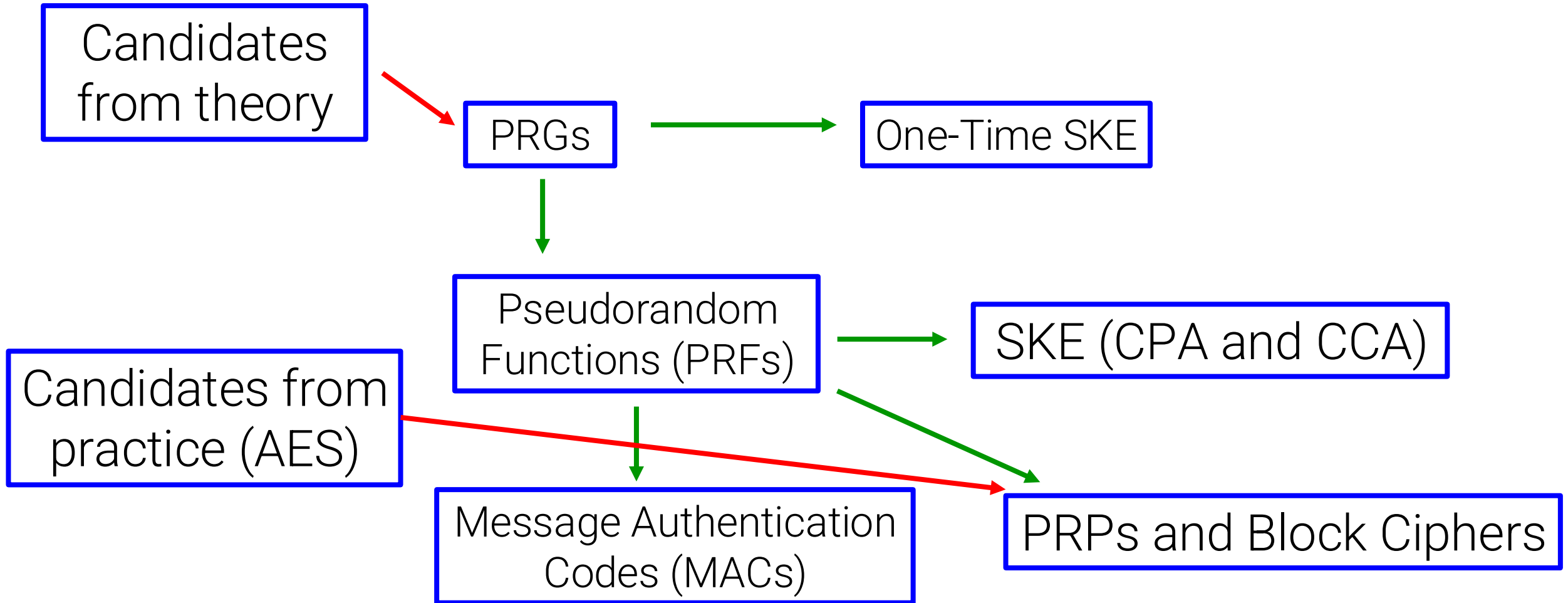
- Problem Set 2 due [Wednesday October 8th](#)
- Problem Set 1 sample solutions were released on Ed.

Recap of class so far

Computationally Secure Cryptography



Computationally Secure Cryptography



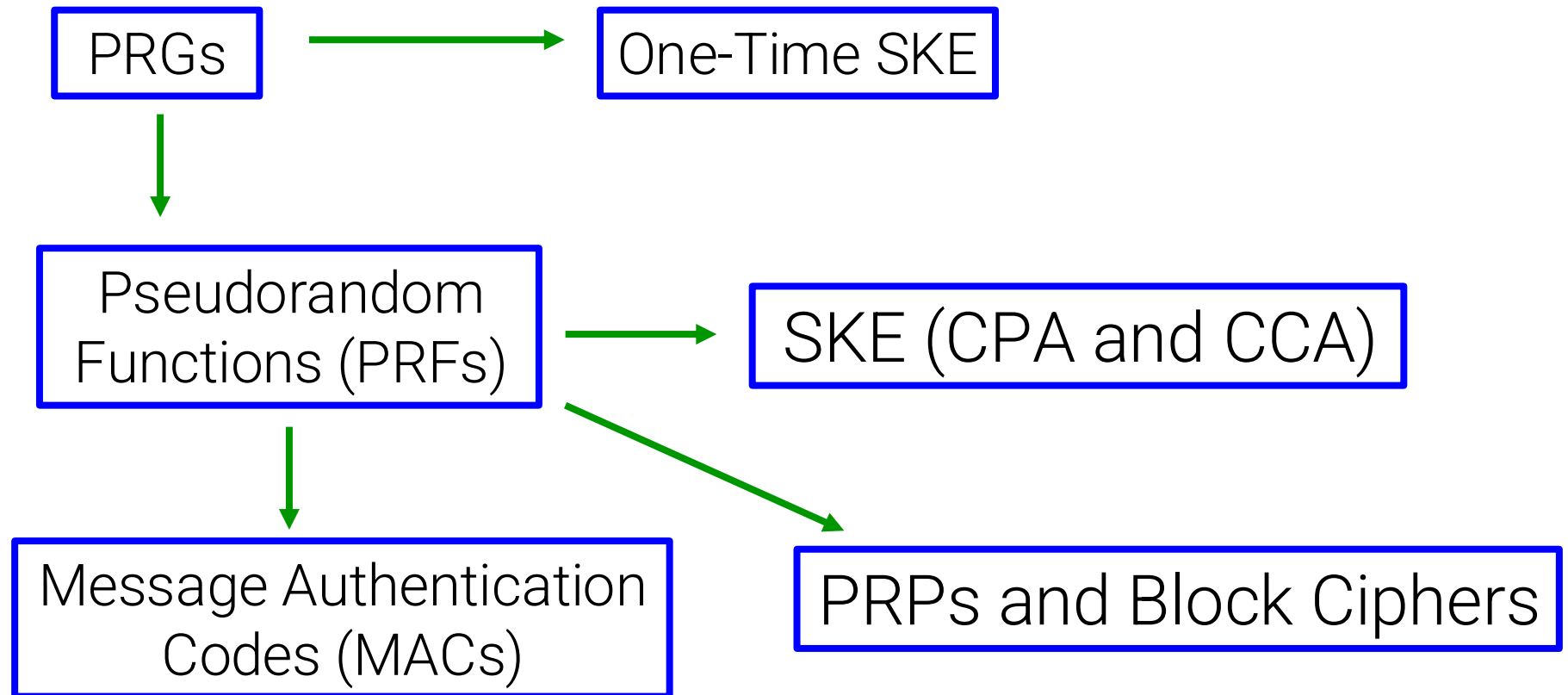
Next two days

- More candidate constructions of cryptography?
- Should we really **believe that PRGs exist**?
- How can we establish computational indistinguishability **without assuming it**?

Computationally Secure Cryptography

These theorems say:

If $X \approx_c Y$
then $X' \approx_c Y'$



Computationally Secure Cryptography

These theorems say:

If $X \approx_c Y$
then $X' \approx_c Y'$

PRGs



Pseudorandom
Functions (PRFs)



Message Authentication
Codes (MACs)



CCA)

Block Ciphers

Why should PRGs or SKE exist?

- Hard to reason about **all polynomial-time distinguishers**
- For our best schemes, attacks boil down to *searching for something*
 - PRGs: given y , find s such that $G(s) = y$
 - SKE: given $ct = \text{Enc}(m_b)$, find the key k
 - (heuristic) $\text{Dec}(k, ct) \in \{m_0, m_1\}$
- Often **we don't know a better way** to break indistinguishability

Wishful thinking:

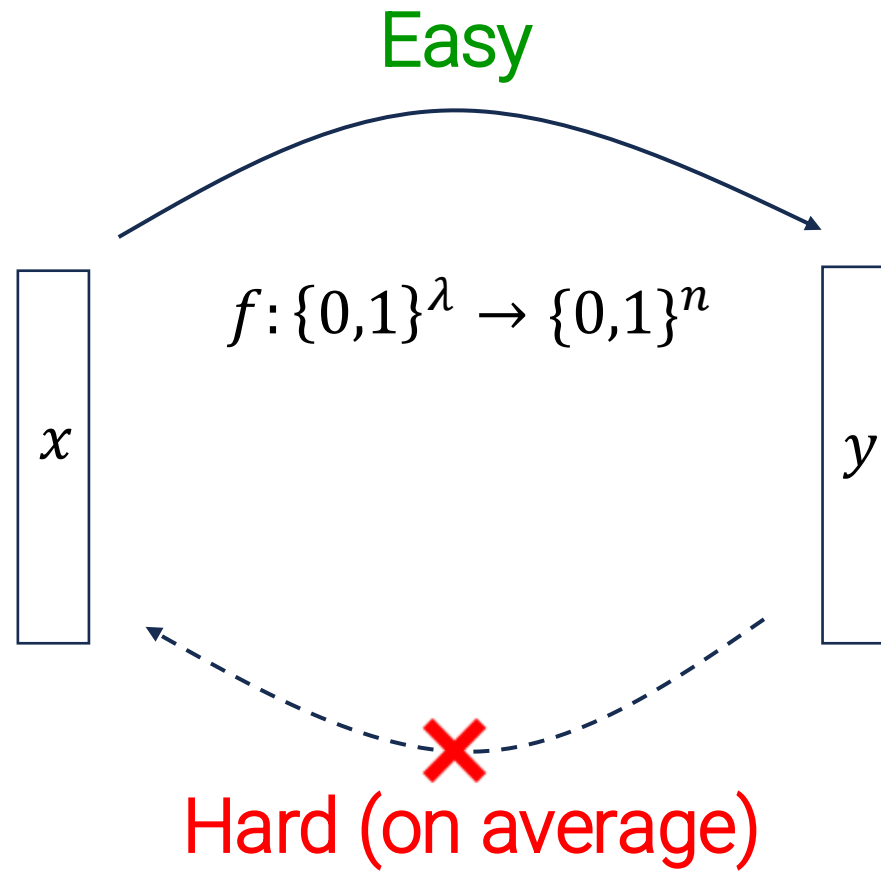
prove PRG secure assuming seed is hard to find!

- Actually possible for certain candidates such as LPN!
- Not true for general G

Informal Theorem:

If **some search problem** is computationally infeasible, **then PRGs exist!**

One-Way Functions

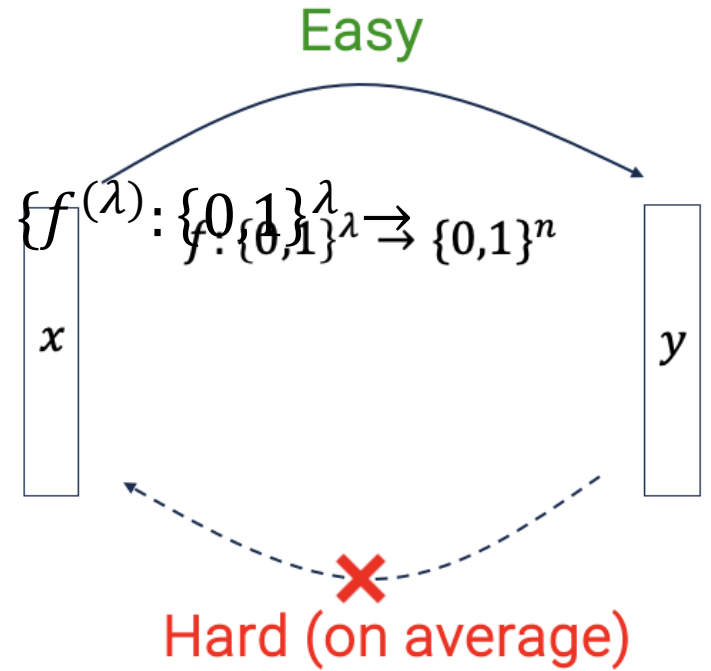


One-Way Functions

Definition: An **efficiently computable** function family $f = \{f^{(\lambda)}: \{0,1\}^\lambda \rightarrow \{0,1\}^n\}_{\lambda}$ is a **one-way function** if:

For all polynomial-time A

$$\Pr_{x \leftarrow \{0,1\}^\lambda} [A(f(x)) = x] = \text{negl}(\lambda)$$

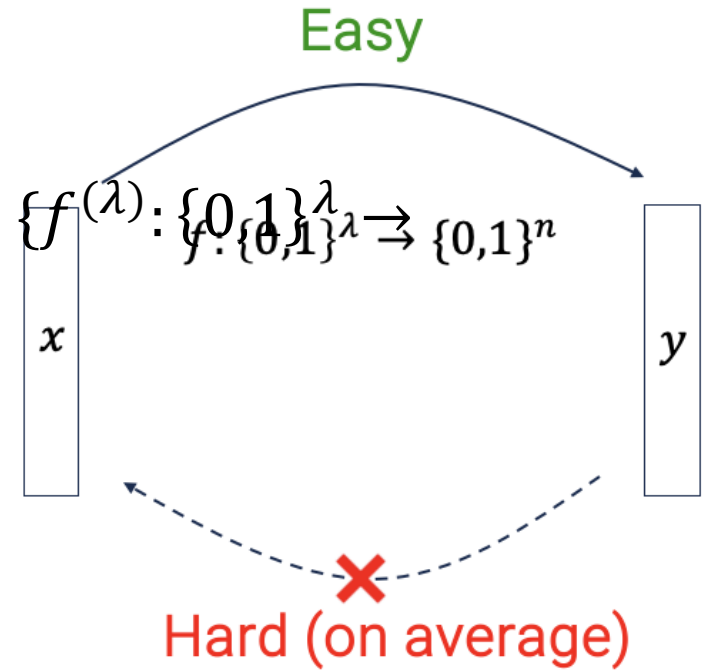


One-Way Functions

Definition: An **efficiently computable** function family $f = \{f^{(\lambda)}: \{0,1\}^\lambda \rightarrow \{0,1\}^n\}_{\lambda}$ is a **one-way function** if:

For all polynomial-time A

$$\Pr_{x \leftarrow \{0,1\}^\lambda} [A(f(x)) = x] = \text{negl}(\lambda)$$



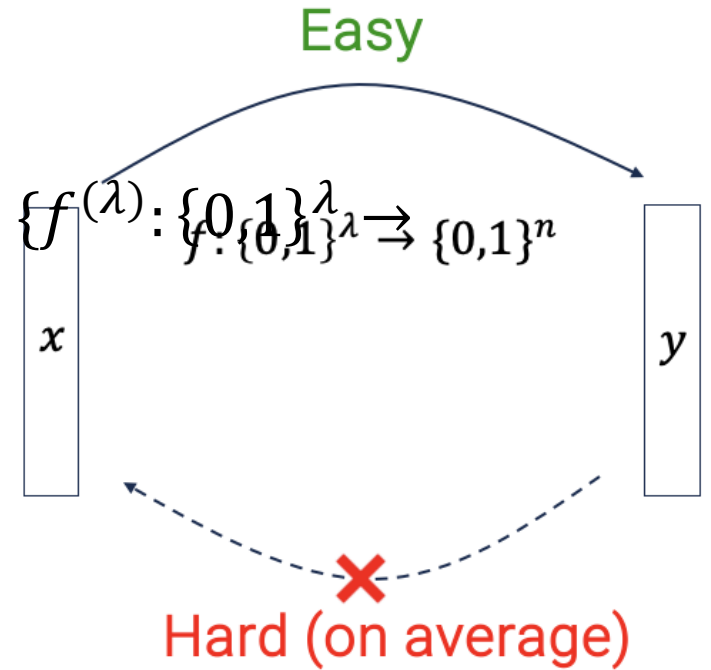
$f(x) \equiv 0$ satisfies this definition

One-Way Functions

Definition: An **efficiently computable** function family $f = \{f^{(\lambda)}: \{0,1\}^\lambda \rightarrow \{0,1\}^n\}_{\lambda}$ is a **one-way function** if:

For all polynomial-time A

$$\Pr_{x \leftarrow \{0,1\}^\lambda} [f(A(f(x))) = f(x)] = \text{negl}(\lambda)$$

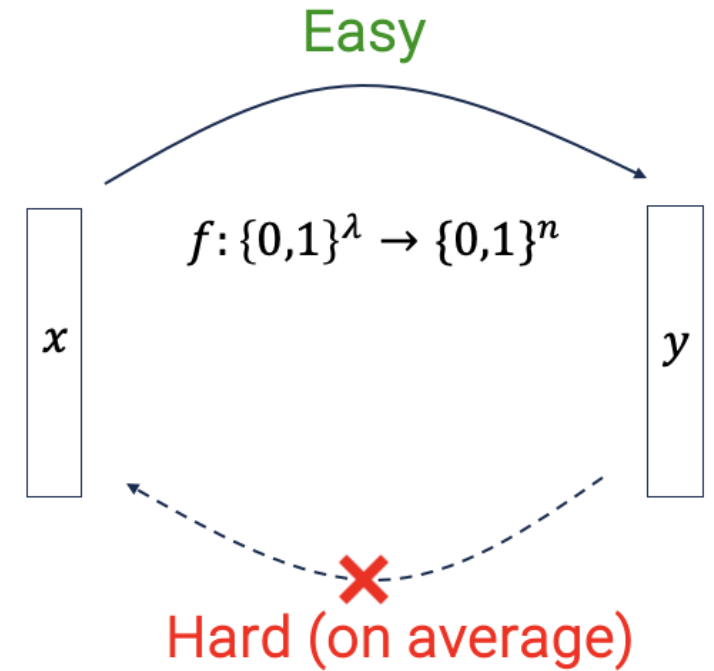


Hard to find **any inverse**.

One-Way Functions

Another view is that OWFs are:

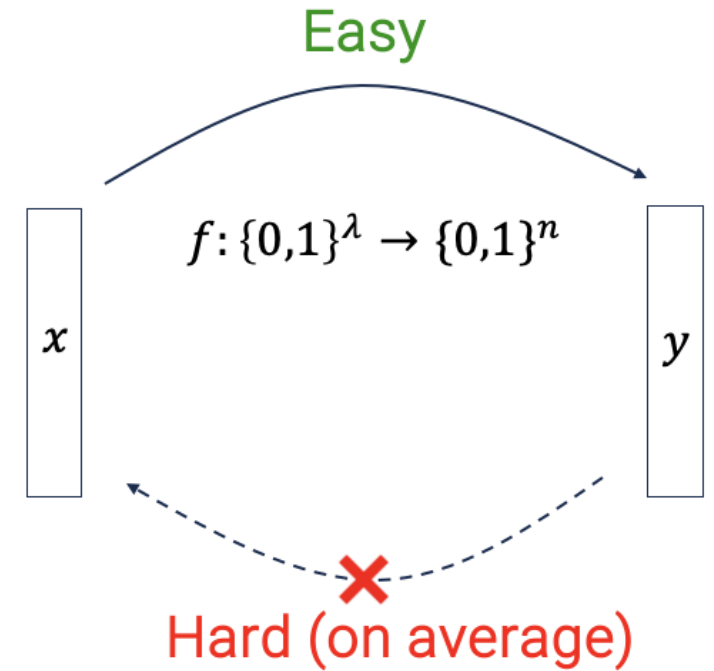
- **hard search problems** (find x' given $y = f(x)$) that you can
 - **generate efficiently** (sample x , compute $y = f(x)$), and
 - **verify efficiently** (given (x', y) , check $y = f(x')$)



One-Way Functions

It's not too hard to see that:

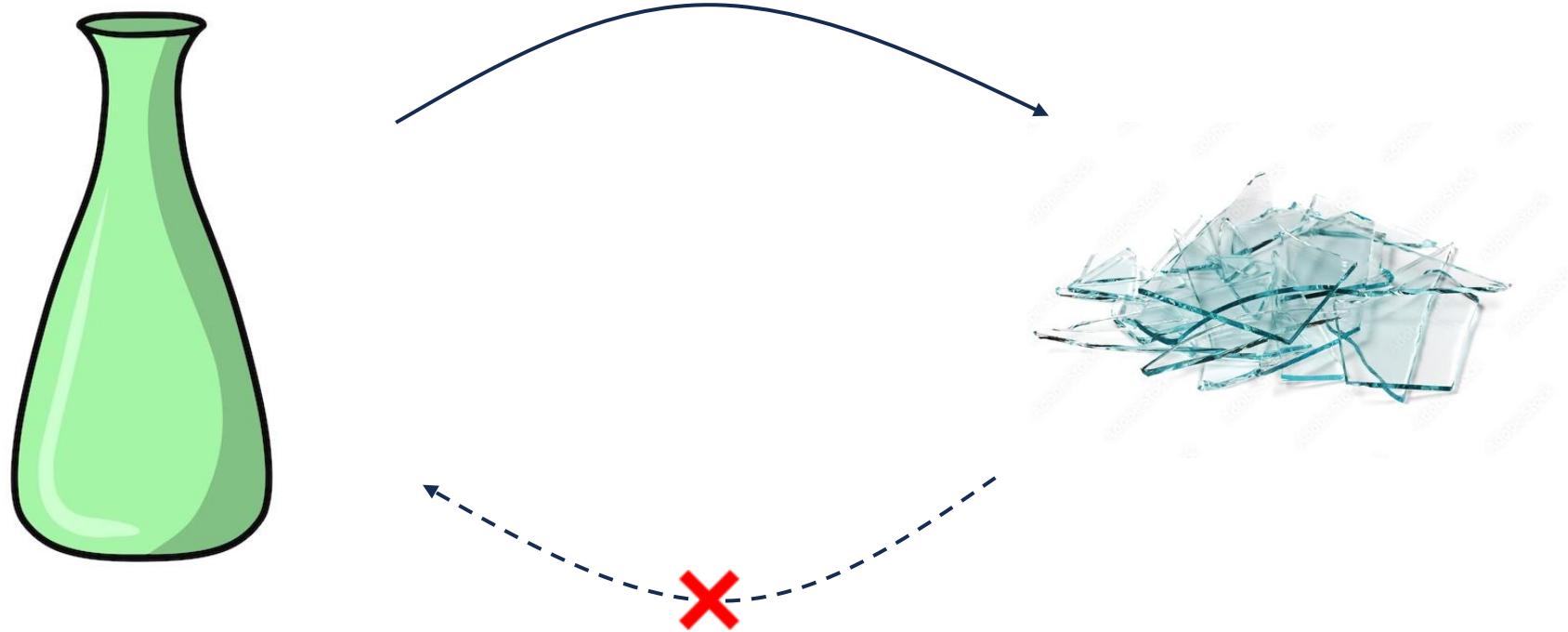
- All PRGs are OWFs (tricky exercise)
- Not all OWFs are PRGs
 - Consider $f(x) = (G(x), 0)$



Conjecture: **One-way functions exist.**

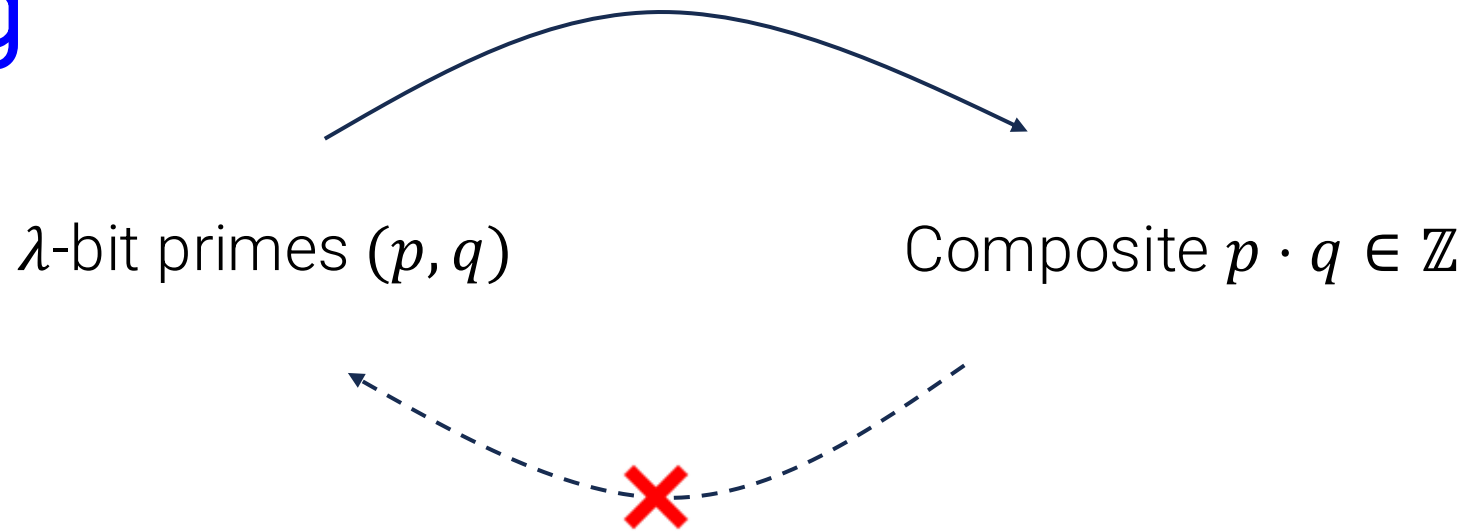
One-way functions are everywhere!

One-Way Functions: Examples?



One-Way Functions: Examples?

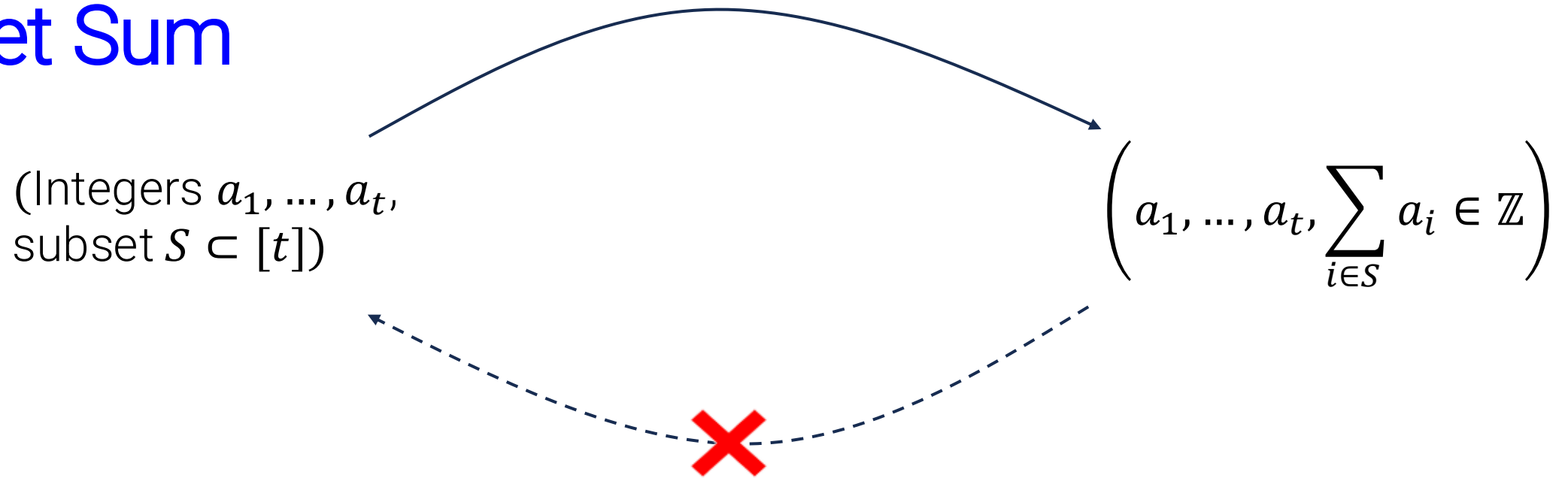
Factoring



- Integer multiplication is **easy**
- Prime factorization is believed to be **hard** (p, q random λ -bit primes)

One-Way Functions: Examples?

Subset Sum



- Computing subset sums is **easy**
- Finding the subset S is believed to be **hard** (uniformly random input)

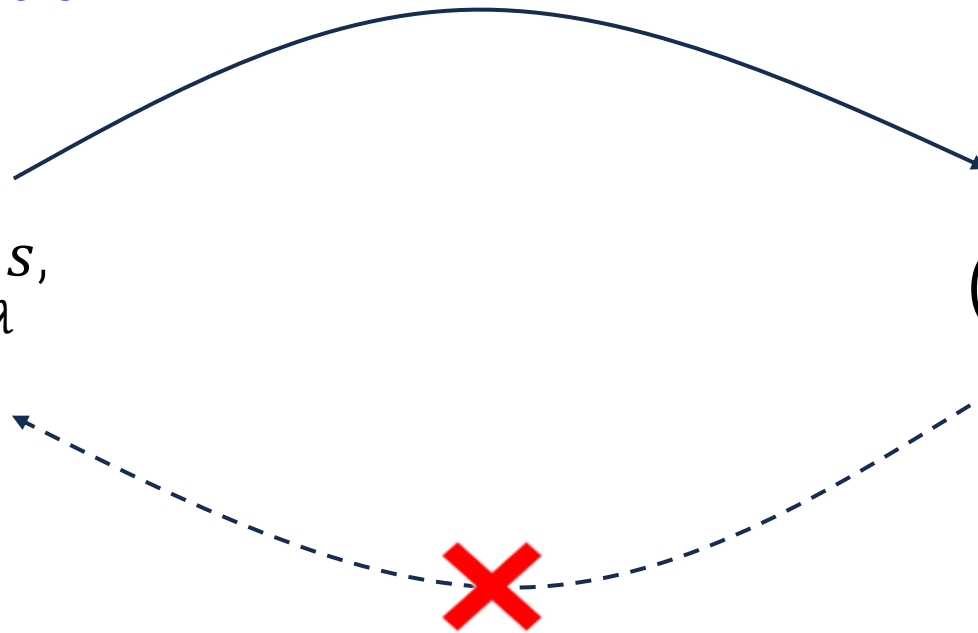
One-Way Functions: Examples?

Random Circuit Evaluation

(less well studied)

Circuit $\mathcal{C}$ of size s ,
input $x \in \{0,1\}^\lambda$

$(\mathcal{C}, \mathcal{C}(x))$



- Evaluating circuits is **easy**
- Inverting circuits probably **hard** (depends on random circuit model)

One-Way Functions

Informal Theorem (we will not prove):

If **any non-trivial cryptography** (in this class) is possible, then **OWFs exist**.

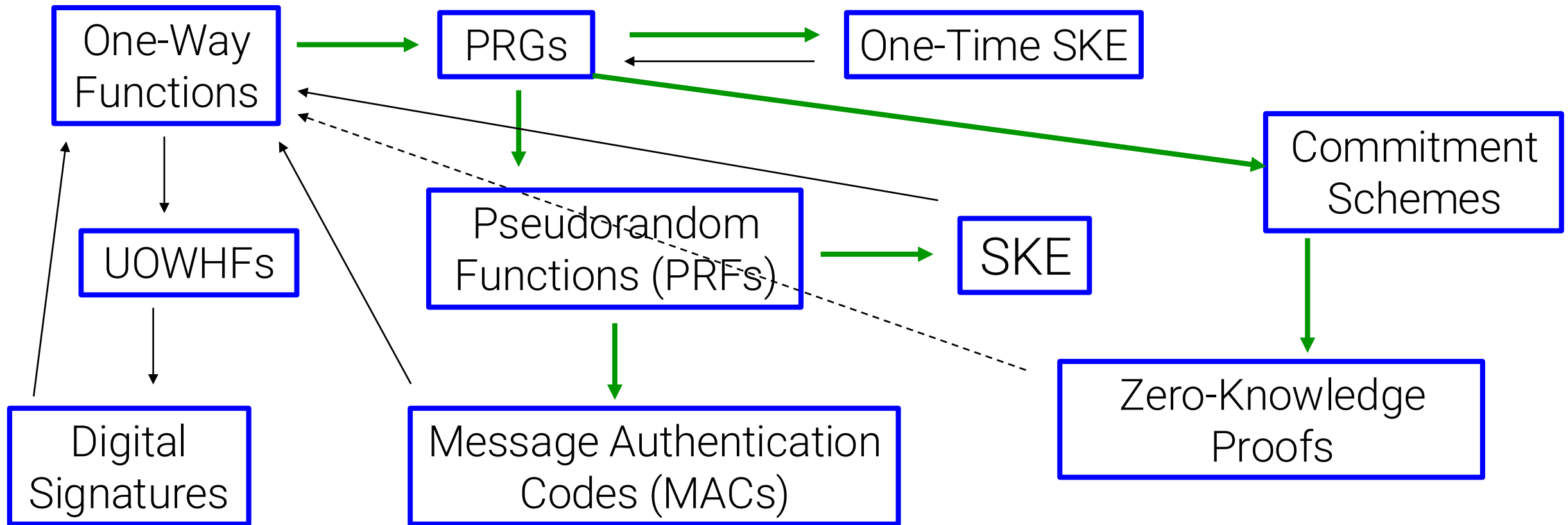
One-Way Functions

Theorem [Håstad-Impagliazzo-Levin-Luby '90] (we will **partially** prove):

If **OWFs exist**, then **PRGs exist!**

Relationships between Cryptographic Primitives

Theorem: the following are **existentially equivalent** (we won't prove this in full)

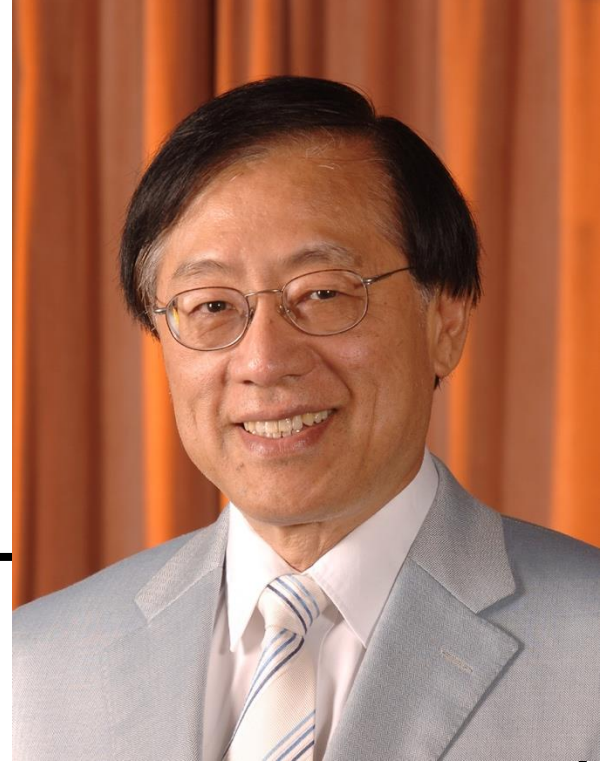


One-Way Functions

Theorem [Håstad-Impagliazzo-Levin-Luby '90] (we will **partially** prove):

If **OWFs exist**, then **PRGs exist!**

One-Way Functions



Theorem [Yao '82, Goldreich-Levin '89]:

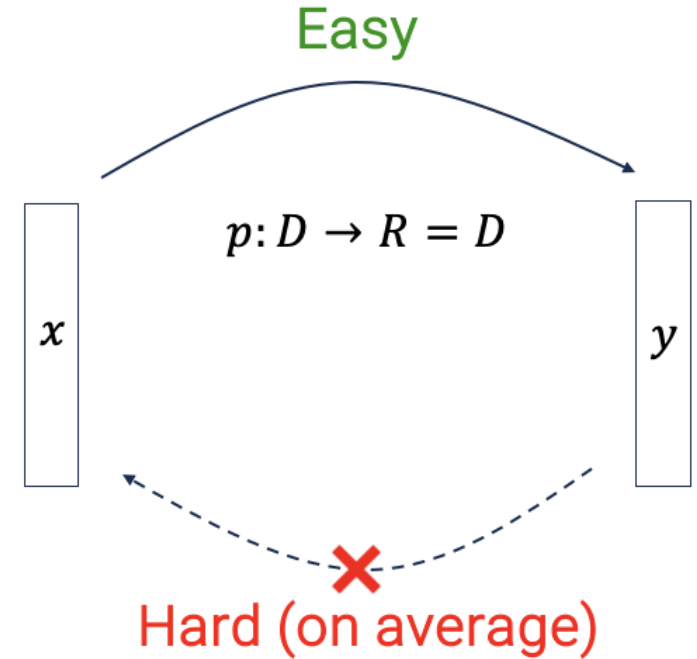
If **One-way permutations exist**, then **PRGs exist!**

One-Way Permutations

Definition: An **efficiently computable** permutation family $p = \{p^{(\lambda)}: \{0,1\}^\lambda \rightarrow \{0,1\}^\lambda\}_\lambda$ is a **one-way permutation** if:

For all polynomial-time A

$$\Pr_{x \leftarrow \{0,1\}^\lambda} [A(p(x)) = x] = \text{negl}(\lambda)$$



PRGs from One-Way Permutations

Theorem [Yao '82, Goldreich-Levin '89]:

If **One-way permutations exist**, then **PRGs exist!**

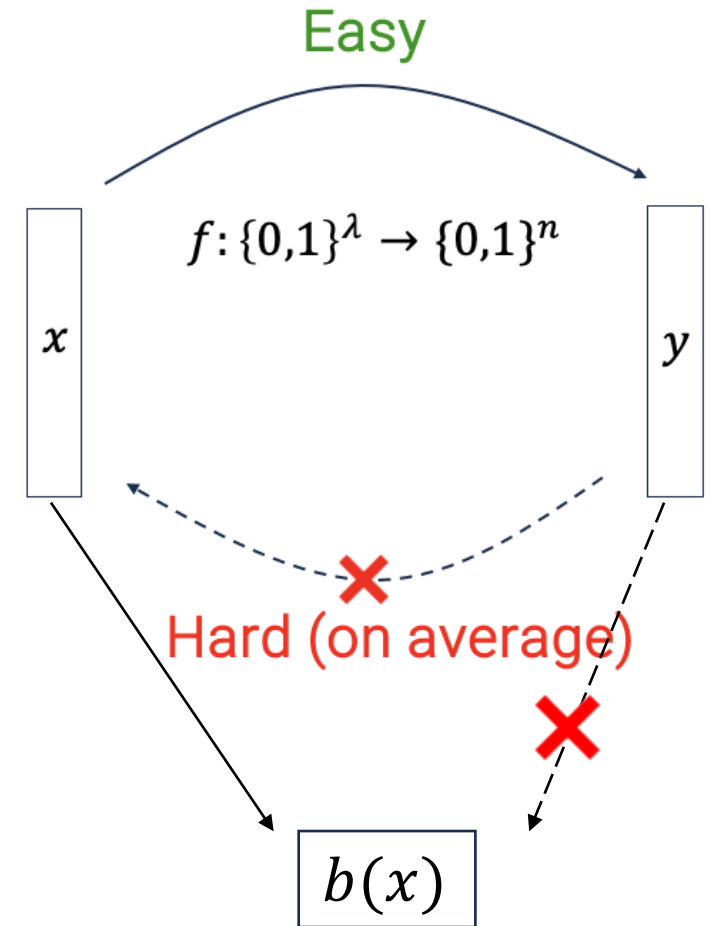
Where does the pseudorandomness come from?

Idea: hard-core bits/predicates

Definition: Given a one-way function f , an **efficiently computable** function family $b = \{b^{(\lambda)}: \{0,1\}^\lambda \rightarrow \{0,1\}\}_\lambda$ is a **hard-core bit (or predicate)** for f if:

For all polynomial-time A

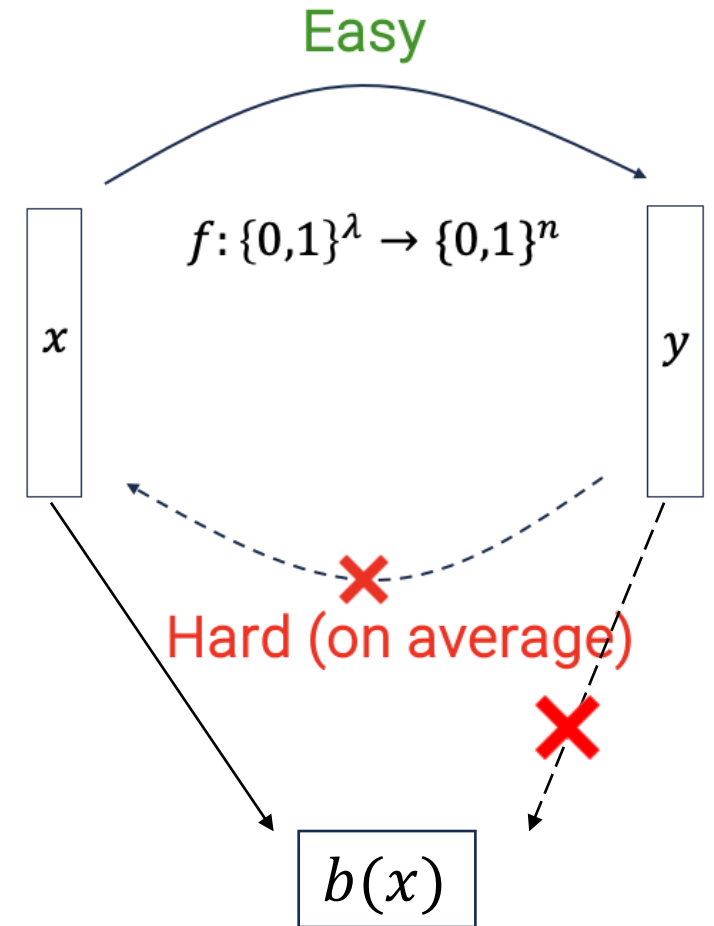
$$\Pr_{x \leftarrow \{0,1\}^\lambda} [A(f(x)) = b(x)] = \frac{1}{2} + \text{negl}(\lambda)$$



Idea: hard-core bits/predicates

Lemma [Yao '82]: Given a one-way **permutation** p and a hard-core predicate b for p , the following construction is a PRG:

$$G(x) = (p(x), b(x)) \in \{0,1\}^{\lambda+1}$$



Idea: hard-core bits/predicates

Lemma: Given a one-way function f and a hard-core predicate b for f , the following two distributions are computationally indistinguishable:

$$(f(x), b(x)) \approx_c (f(x), b)$$

Proof: For any efficient distinguisher D ,

$$\mathbf{E}[D(f(x), b)] = \frac{1}{2} \cdot \mathbf{E}[D(f(x), b(x))] + \frac{1}{2} \cdot \mathbf{E}[D(f(x), 1 - b(x))]$$

Idea: hard-core bits/predicates

Lemma: Given a one-way function f and a hard-core predicate b for f , the following two distributions are computationally indistinguishable:

$$(f(x), b(x)) \approx_c (f(x), b)$$

Proof: For any efficient distinguisher D , if $\mathbf{E}[D(f(x), b(x))] > \mathbf{E}[D(f(x), b)] + \epsilon$,

$$\mathbf{E}[D(f(x), b(x))] > \mathbf{E}[D(f(x), 1 - b(x))] + 2\epsilon$$

Idea: hard-core bits/predicates

Lemma: Given a one-way function f and a hard-core predicate b for f , the following two distributions are computationally indistinguishable:

$$(f(x), b(x)) \approx_c (f(x), b)$$

Proof: For any efficient distinguisher D , if $\mathbf{E}[D(f(x), b(x))] > \mathbf{E}[D(f(x), b)] + \epsilon$, here's how you can guess $b(x)$ given $f(x)$:

$$A(y) := b \text{ if } D(y, b) = 1$$

Idea: hard-core bits/predicates

Lemma: Given a one-way function f and a hard-core predicate b for f , the following two distributions are computationally indistinguishable:

$$(f(x), b(x)) \approx_c (f(x), b)$$

Proof: For any efficient distinguisher D , if $\mathbf{E}[D(f(x), b(x))] > \mathbf{E}[D(f(x), b)] + \epsilon$,

$$\Pr[A(f(x)) = b(x)] = \frac{1}{2} \cdot \mathbf{E}[D(f(x), b(x))] + \frac{1}{2} \cdot \mathbf{E}[1 - D(f(x), 1 - b(x))]$$

Idea: hard-core bits/predicates

Lemma: Given a one-way function f and a hard-core predicate b for f , the following two distributions are computationally indistinguishable:

$$(f(x), b(x)) \approx_c (f(x), b)$$

Proof: For any efficient distinguisher D , if $\mathbf{E}[D(f(x), b(x))] > \mathbf{E}[D(f(x), b)] + \epsilon$,

$$\Pr[A(f(x)) = b(x)] = \frac{1}{2} + \frac{1}{2} \cdot (\mathbf{E}[D(f(x), b(x))] - \mathbf{E}[D(f(x), 1 - b(x))])$$

Idea: hard-core bits/predicates

Lemma: Given a one-way function f and a hard-core predicate b for f , the following two distributions are computationally indistinguishable:

$$(f(x), b(x)) \approx_c (f(x), b)$$

Proof: For any efficient distinguisher D , if $\mathbf{E}[D(f(x), b(x))] > \mathbf{E}[D(f(x), b)] + \epsilon$,

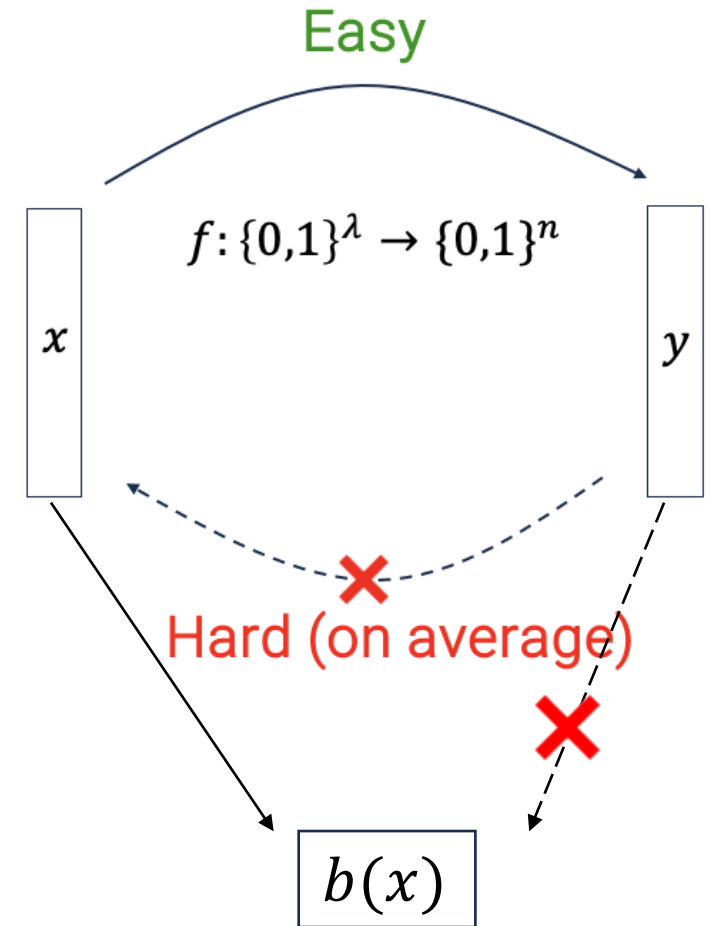
$$\Pr[A(f(x)) = b(x)] > \frac{1}{2} + \epsilon$$

Idea: hard-core bits/predicates

Lemma [Yao '82]: Given a one-way **permutation** p and a hard-core predicate b for p , the following construction is a PRG:

$$G(x) = (p(x), b(x)) \in \{0,1\}^{\lambda+1}$$

Proof: $G(\textcolor{red}{x}) \approx_c (p(\textcolor{red}{x}), \textcolor{green}{b}) \equiv (\textcolor{red}{r}, \textcolor{green}{b})$



PRGs from One-Way Permutations

Question:

Does **every OWF** have a hard-core predicate?

Idea: what about $b(x) = x_1$?

PRGs from One-Way Permutations

Question:

Does **every OWF** have a hard-core predicate?

Idea: what about $b(x) = x_i$ for some i ?

PRGs from One-Way Permutations

Question:

Does **every OWF** have a hard-core predicate?

Open problem!

PRGs from One-Way Permutations

Theorem [Goldreich-Levin '89]:

Every OWF has a randomized hard-core predicate

Given OWF f , define:

$$f'(x, r) = (f(x), r)$$

$$b(x, r) = \langle x, r \rangle = \sum_i x_i r_i \bmod 2$$

PRGs from One-Way Permutations

Theorem [Goldreich-Levin '89]:

For every OWF f , $b(x, r)$ is a hard-core predicate for f'

Given OWF f , define:

$$f'(x, r) = (f(x), r)$$

$$b(x, r) = \langle x, r \rangle = \sum_i x_i r_i \bmod 2$$

PRGs from One-Way Permutations

Theorem [Goldreich-Levin '89]:

For every OWF f , $b(x, r)$ is a hard-core predicate for f'

Given $(f(x), r)$, hard to predict $\langle x, r \rangle$

$$(f(x), r, \langle x, r \rangle) \approx_c (f(x), r, b)$$

PRGs from One-Way Permutations

Theorem [Goldreich-Levin '89]:

For every OWF f , $b(x, r)$ is a hard-core predicate for f'

Corollary [Goldreich-Levin '89]:

For every OWP p , $G(x, r) = (p(x), r, b(x, r))$ is a PRG!

PRGs from One-Way Permutations

Theorem [Goldreich-Levin '89]:

For every OWF f , $b(x, r)$ is a hard-core predicate for f'

Proof: start on the board (if time)